

Automated Calibration Target Extraction & Joint Bayesian Inference

QSP-LLM-Workflows Progress Update

Joel Eliason

Popel Lab

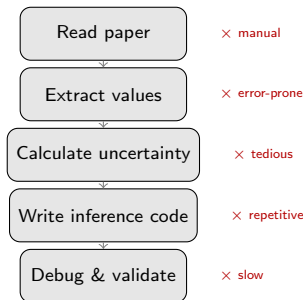
January 19, 2026

The Challenge

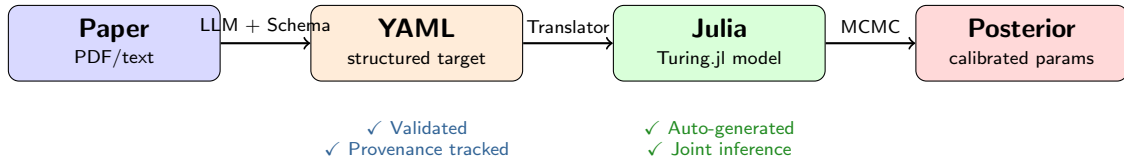
Problem: QSP model calibration requires

- Manual extraction of data from papers
- Uncertainty quantification for each parameter
- Translation to inference code
- Validation of extracted values

Current state: Error-prone, time-consuming, poorly documented



Our Approach: Automated Pipeline

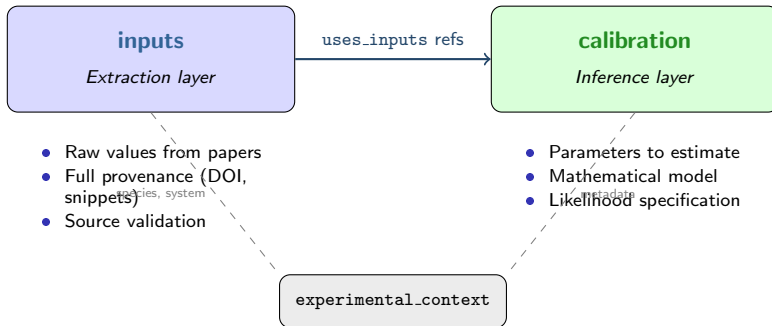


Key benefits:

- Pydantic validators catch errors *before* inference
- Full provenance: every value traces back to source text
- Automatic Julia/Turing.jl code generation with parameter sharing

Solution: SimplifiedIsolatedTarget Schema

Key Innovation: Pydantic schema separating *what* from *how*



inputs: Extraction with Provenance

```
inputs:
- name: pdgfold_mean
  value: 4.37
  units: dimensionless
  input_type: direct_measurement # vs proxy, inferred, assumed
  role: target # vs auxiliary, initial_condition
  source_ref: Schneider2001_PSC_PDGF
  source_location: Abstract
  value_snippet: "Cell proliferation (4.37 +/- 0.49-fold of control)."
```

```
- name: pdgfold_sd
  value: 0.49
  units: dimensionless
  input_type: direct_measurement
  role: auxiliary
  source_ref: Schneider2001_PSC_PDGF
```

Input types: direct_measurement, proxy_measurement, inferred_estimate, assumed_value

Roles: target (in likelihood), auxiliary (supporting), initial_condition (ODE IC)

calibration.parameters: Prior Specification

```
calibration:
  parameters:
    - name: k_apsc_prolif
      units: 1/day
      prior:
        distribution: lognormal
        mu: 0.0      #  $\log(1.0)$  = center at 1/day
        sigma: 1.0   # ~68% between 0.37 and 2.7/day
        rationale: |
          Wide log-normal prior centered at 1/day. A 4-fold increase
          in 1 day implies  $k \sim \ln(4) \sim 1.4/\text{day}$ .
```

Supported distributions:

- `lognormal( $\mu$ ,  $\sigma$ )` — positive parameters (rates, concentrations)
- `normal( $\mu$ ,  $\sigma$ )` — unconstrained parameters
- `uniform( $\text{lower}$ ,  $\text{upper}$ )` — bounded parameters
- `half_normal( $\sigma$ )` — positive with mode at 0

Built-in ODE models:

- exponential_growth: $\dot{y} = ky$
- first_order_decay: $\dot{y} = -ky$
- two_state: $A \xrightarrow{k} B$
- saturation: $\dot{y} = k(1 - y)$
- logistic: $\dot{y} = ky(1 - y/K)$
- michaelis_menten: $\dot{y} = -V_{max}y/(K_m + y)$

Non-ODE models:

- direct_conversion: $k = \ln(2)/t_{1/2}$
- direct_fit: Hill, linear, exponential
- custom: user-provided code

```
model:
  type: exponential_growth
  rate_constant: k_apsc_prolif
  data_rationale: |
    BrdU assay measures proliferation
    as fold-change over ~24h under
    constant PDGF stimulus.
  submodel_rationale: |
    Full model: k_apsc_prolif * aPSC
    * (PDGF/(PDGF_50 + PDGF))
    At 50 ng/mL PDGF >> PDGF_50,
    reduces to exponential growth.
```

calibration.measurements: Likelihood & Observables

```
measurements:
- name: proliferation_fold_change
  observable:
    type: identity
    state_variables: [aPSC_rel]
  units: dimensionless
  uses_inputs: [pdgf_fold_mean, pdgf_fold_sd]
  evaluation_points: [1.0] # day
  sample_size: 3
  distribution_code: |
    def derive_distribution(inputs, ureg):
        mean = inputs['pdgf_fold_mean'].magnitude
        sd = inputs['pdgf_fold_sd'].magnitude
        # Monte Carlo to get median and 95% CI
        ...
        return {'median': [4.34], 'ci95': [[3.4, 5.3]], 'units': '...'}
  likelihood:
    distribution: lognormal
    rationale: Fold-changes are positive and multiplicative.
```


experimental_context: Metadata for Generalization

```
experimental_context:
  species: rat
  system: in_vitro_primary_cells
  indication: PDAC
  cell_types:
    - name: pancreatic stellate cell
      phenotype: activated (alpha-SMA positive)
      isolation_method: Isolated from normal Wistar rat pancreas
  culture_conditions:
    culture_type: 2d_monolayer
```

Why this matters:

- Enables filtering by species/system for meta-analysis
- Documents translation assumptions (rat $\rightarrow$ human)
- Supports uncertainty inflation for cross-species extrapolation

Built-in Pydantic Validators

10 validators ensure data integrity and prevent hallucinations:

Reference Validation

- `validate_input_refs` — all `uses_inputs` exist
- `validate_source_refs` — all `source_ref` match `source_tags`
- `validate_parameter_roles` — model params exist
- `validate_observable_state_vars`

Content Validation

- `validate_doi_resolution` — DOI resolves via CrossRef
- `validate_input_values_in_snippets` — anti-hallucination check
- `validate_units_are_valid_pint`
- `validate_custom_code_syntax` — AST parsing
- `validate_span_ordering`
- `validate_ode_model_requirements`

Example: Anti-hallucination check

If `value_snippet` = "proliferation (4.37 +/- 0.49-fold)" but `value` = 5.2, validator raises error.

Automatic Julia/Turing.jl Code Generation

Python YAML:

```
calibration:
  parameters:
    - name: k_apsc_prolif
      prior:
        distribution: lognormal
        mu: 0.0
        sigma: 1.0
  model:
    type: exponential_growth
    rate_constant: k_apsc_prolif
```

Generated Julia:

```
function ode!(du, u, p, t)
    k_apsc_prolif = p[1]
    du[1] = k_apsc_prolif * u[1]
end

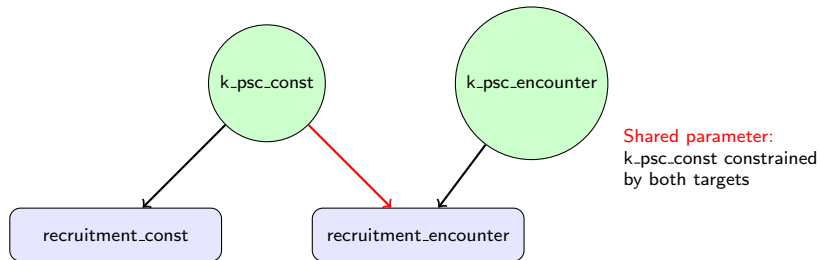
@model function calibration(data)
    # Prior
    k_apsc_prolif ~ LogNormal(0.0, 1.0)

    # Simulate
    sol = solve(ODEProblem(ode!, ...))
    pred = sol[1, end]

    # Likelihood
    data.obs ~ Normal(pred, data.sigma)
end
```

Key feature: **Joint inference** — parameters shared across targets are automatically identified and estimated once.

Joint Inference: Parameter Sharing



10 targets → **10 parameters** (1 shared: k_psc_const)

Joint posterior captures correlations and improves identifiability.

Joint Inference Results: 10 Parameters

Parameter	Posterior Median	Units	R-hat	ESS
k_ECM_apsc_sec	8.74×10^{-10}	mg/(cell·day)	1.001	3839
k_psc_activation	2.58	1/day	1.000	4074
k_apsc_death	0.091	1/day	1.002	4020
k_qpsc_death	0.178	1/day	1.002	3802
k_apsc_prolif	1.44	1/day	1.001	4590
k_psc_const	3.49×10^7	cell/mL	1.002	2670
k_psc_encounter	2.54×10^5	cell/(mL·day)	1.001	2388
p_T_kill_per_contact	0.150	–	1.000	4023
k_TGFb_sec_apsc	3.75×10^{-10}	nM·L/(cell·day)	1.002	3282
R50_Treg	0.411	–	1.000	4571

All parameters converged: R-hat $\approx$ 1.0, ESS > 2000

New target: R50_Treg (Treg:Teff ratio for 50% suppression) = 0.41

Posterior Predictive Diagnostics

Target	Observed	Predicted	Z-score	90%	95%
ecm_secretion_apsc	6.48×10^{-10}	8.74×10^{-10}	-0.69	✓	✓
psc_activation_rate	0.950	0.994	-1.19	✓	✓
psc_death_activated	0.086	0.078	0.38	✓	✓
psc_death_quiescent	0.198	0.178	0.42	✓	✓
psc_proliferation	4.34	4.23	0.22	✓	✓
psc_recruitment_const	3.72×10^7	3.49×10^7	0.15	✓	✓
tcell_kill_probability	0.150	0.150	-0.01	✓	✓
tgfb_secretion_apsc	4.25×10^{-10}	3.75×10^{-10}	0.27	✓	✓
treg_suppression_r50	0.507	0.411	0.66	✓	✓

Summary Statistics:

- Mean $|Z| = 0.44$ (ideal: <1.0) ✓
- Max $|Z| = 1.19$ (ideal: <2.0) ✓
- RMSE = 0.56 (ideal: ~ 1.0) ✓

Posterior Coverage:

- 90% CI: 100% coverage (expected: 90%)
 - 95% CI: 100% coverage (expected: 95%)
- Well-calibrated uncertainties

Summary & Next Steps

What we built:

- **SimplifiedIsolatedTarget** — Pydantic schema separating extraction from inference
- **10 built-in validators** — DOI resolution, anti-hallucination, unit checking
- **Julia translator** — Automatic Turing.jl code generation with joint inference
- **10 calibration targets** for PDAC stromal biology

Results:

- All 10 parameters converged ($R\text{-hat} \approx 1.0$)
- Excellent posterior predictive metrics (mean $|Z| = 0.44$, 100% coverage)
- New target: $R50_{Treg} = 0.41$ (Treg suppression)

Next steps:

- Expand to full PDAC immune module (CD8, MDSC, macrophages)
- LLM-assisted extraction pipeline with human-in-the-loop
- Integration with full QSP model for virtual population generation

Questions?

Code: `github.com/popellab/qsp-llm-workflows`